

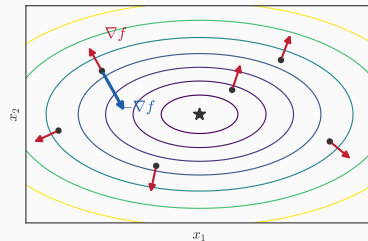
Otimização

3.3.1 Método do gradiente (*steepest descent*, método de Cauchy)

José Firmino Aguilar Madeira

Se só conhecemos $\nabla f(\mathbf{x}_k)$, para onde ir?

O 3.3 terminou com esta pergunta. Do 3.1: o gradiente é normal à curva de nível e aponta para a maior **subida** local.

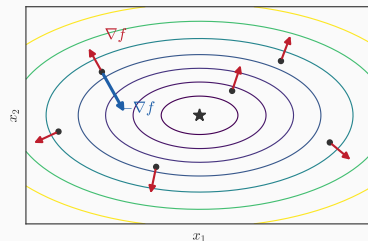


A figura do 3.1: ∇f (vermelho) e $-\nabla f$ (azul).

Se só conhecemos $\nabla f(\mathbf{x}_k)$, para onde ir?

O 3.3 terminou com esta pergunta. Do 3.1: o gradiente é normal à curva de nível e aponta para a maior **subida** local.

Entre as direções com o mesmo comprimento,
 $\nabla f^T \mathbf{d} \geq -\|\nabla f\| \|\mathbf{d}\|$ (Cauchy–Schwarz), com igualdade em $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$: a descida **mais íngreme localmente**.



A figura do 3.1: ∇f (vermelho) e $-\nabla f$ (azul).

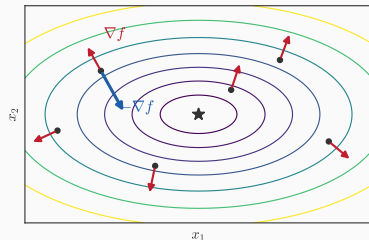
Se só conhecemos $\nabla f(\mathbf{x}_k)$, para onde ir?

O 3.3 terminou com esta pergunta. Do 3.1: o gradiente é normal à curva de nível e aponta para a maior **subida** local.

Entre as direções com o mesmo comprimento,
 $\nabla f^T \mathbf{d} \geq -\|\nabla f\| \|\mathbf{d}\|$ (Cauchy–Schwarz), com igualdade em $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$: a descida **mais íngreme localmente**.

Atenção ao nome

Steepest descent = a direção mais íngreme **no ponto**, para $\|\mathbf{d}\|$ fixa — não “o algoritmo que chega mais depressa ao mínimo”.



A figura do 3.1: ∇f (vermelho) e $-\nabla f$ (azul).

Se só conhecemos $\nabla f(\mathbf{x}_k)$, para onde ir?

O 3.3 terminou com esta pergunta. Do 3.1: o gradiente é normal à curva de nível e aponta para a maior **subida** local.

Entre as direções com o mesmo comprimento,
 $\nabla f^T \mathbf{d} \geq -\|\nabla f\| \|\mathbf{d}\|$ (Cauchy–Schwarz), com igualdade em $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$: a descida **mais íngreme localmente**.

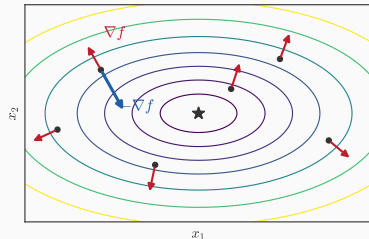
Atenção ao nome

Steepest descent = a direção mais íngreme **no ponto**, para $\|\mathbf{d}\|$ fixa — não “o algoritmo que chega mais depressa ao mínimo”.

E **quanto andar**? Isso é outra decisão: o passo α_k , pela pesquisa em linha do cap. 2.

3.3 escolhe $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$

o cap. 2 escolhe α_k

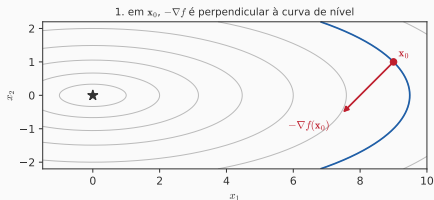


A figura do 3.1: ∇f (vermelho) e $-\nabla f$ (azul).

Uma iteração à mão — e nasce a perpendicularidade

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\nabla f = (x_1, 9x_2)^T$, mínimo em $(0,0)$;
 $\mathbf{x}_0 = (9,1)$.

1. **Direção:** $\mathbf{g}_0 = \nabla f(\mathbf{x}_0) = (9,9)$, $\mathbf{d}_0 = -\mathbf{g}_0 = (-9,-9)$.



Uma iteração à mão — e nasce a perpendicularidade

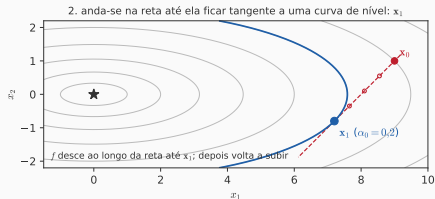
$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\nabla f = (x_1, 9x_2)^T$, mínimo em $(0, 0)$;
 $\mathbf{x}_0 = (9, 1)$.

1. **Direção:** $\mathbf{g}_0 = \nabla f(\mathbf{x}_0) = (9, 9)$, $\mathbf{d}_0 = -\mathbf{g}_0 = (-9, -9)$.

2. **Quanto andar:**

$$\varphi(\alpha) = f(\mathbf{x}_0 + \alpha\mathbf{d}_0) = \frac{1}{2}[(9 - 9\alpha)^2 + 9(1 - 9\alpha)^2],$$

$$\varphi'(\alpha) = 810\alpha - 162 = 0 \Rightarrow \alpha_0 = 0,2.$$



Uma iteração à mão — e nasce a perpendicularidade

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\nabla f = (x_1, 9x_2)^T$, mínimo em $(0, 0)$;
 $\mathbf{x}_0 = (9, 1)$.

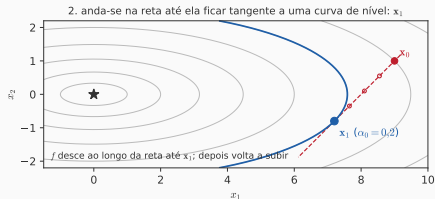
1. **Direção:** $\mathbf{g}_0 = \nabla f(\mathbf{x}_0) = (9, 9)$, $\mathbf{d}_0 = -\mathbf{g}_0 = (-9, -9)$.

2. **Quanto andar:**

$$\varphi(\alpha) = f(\mathbf{x}_0 + \alpha \mathbf{d}_0) = \frac{1}{2}[(9 - 9\alpha)^2 + 9(1 - 9\alpha)^2],$$

$$\varphi'(\alpha) = 810\alpha - 162 = 0 \Rightarrow \alpha_0 = 0,2.$$

3. **Novo ponto:** $\mathbf{x}_1 = \mathbf{x}_0 + 0,2 \mathbf{d}_0 = (7,2; -0,8)$, $f = 28,8$
(era 45).



Uma iteração à mão — e nasce a perpendicularidade

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\nabla f = (x_1, 9x_2)^T$, mínimo em $(0, 0)$;
 $\mathbf{x}_0 = (9, 1)$.

1. **Direção:** $\mathbf{g}_0 = \nabla f(\mathbf{x}_0) = (9, 9)$, $\mathbf{d}_0 = -\mathbf{g}_0 = (-9, -9)$.

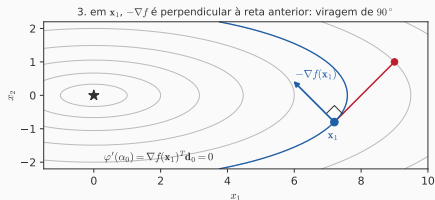
2. **Quanto andar:**

$$\varphi(\alpha) = f(\mathbf{x}_0 + \alpha \mathbf{d}_0) = \frac{1}{2}[(9 - 9\alpha)^2 + 9(1 - 9\alpha)^2],$$

$$\varphi'(\alpha) = 810\alpha - 162 = 0 \Rightarrow \alpha_0 = 0,2.$$

3. **Novo ponto:** $\mathbf{x}_1 = \mathbf{x}_0 + 0,2 \mathbf{d}_0 = (7,2; -0,8)$, $f = 28,8$
(era 45).

4. **Surpresa:** $\mathbf{g}_1 = (7,2; -7,2)$ e
 $\nabla f(\mathbf{x}_1)^T \mathbf{d}_0 = -64,8 + 64,8 = 0$.



Uma iteração à mão — e nasce a perpendicularidade

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\nabla f = (x_1, 9x_2)^T$, mínimo em $(0, 0)$;
 $\mathbf{x}_0 = (9, 1)$.

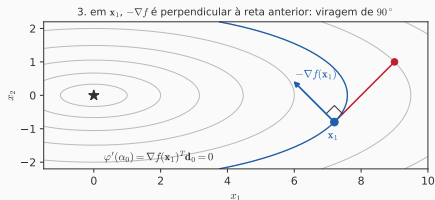
1. **Direção:** $\mathbf{g}_0 = \nabla f(\mathbf{x}_0) = (9, 9)$, $\mathbf{d}_0 = -\mathbf{g}_0 = (-9, -9)$.

2. **Quanto andar:**

$$\varphi(\alpha) = f(\mathbf{x}_0 + \alpha \mathbf{d}_0) = \frac{1}{2}[(9 - 9\alpha)^2 + 9(1 - 9\alpha)^2],$$
$$\varphi'(\alpha) = 810\alpha - 162 = 0 \Rightarrow \alpha_0 = 0,2.$$

3. **Novo ponto:** $\mathbf{x}_1 = \mathbf{x}_0 + 0,2 \mathbf{d}_0 = (7,2; -0,8)$, $f = 28,8$ (era 45).

4. **Surpresa:** $\mathbf{g}_1 = (7,2; -7,2)$ e
 $\nabla f(\mathbf{x}_1)^T \mathbf{d}_0 = -64,8 + 64,8 = 0$.



Porquê: pesquisa em linha exata

No mínimo de φ : $\varphi'(\alpha_k) = \mathbf{g}_{k+1}^T \mathbf{d}_k = 0$. Como

$$\mathbf{d}_k = -\mathbf{g}_k: \quad \boxed{\mathbf{g}_{k+1}^T \mathbf{g}_k = 0}$$

Vale para qualquer f diferenciável, desde que α_k minimize φ (não é específico das quadráticas): **gradientes consecutivos ortogonais**. Em 3.3.3 procuraremos uma noção melhor — conjugação em relação a $H = \nabla^2 f$.

O algoritmo

Método do gradiente: $\mathbf{x}_0, \varepsilon_g, \varepsilon_x, k_{\max}$

```
1:  $k \leftarrow 0$ 
2: loop
3:    $\mathbf{g}_k \leftarrow \nabla f(\mathbf{x}_k)$ 
4:   if  $\|\mathbf{g}_k\| \leq \varepsilon_g$  or  $k \geq k_{\max}$  then return  $\mathbf{x}_k$ 
5:   end if
6:    $\alpha_k \leftarrow \arg \min_{\alpha > 0} f(\mathbf{x}_k - \alpha \mathbf{g}_k)$   $\triangleright$  pesquisa em linha
7:    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha_k \mathbf{g}_k$ 
8:   if  $\|\mathbf{x}_{k+1} - \mathbf{x}_k\| / \max(1, \|\mathbf{x}_k\|) \leq \varepsilon_x$  then return  $\mathbf{x}_{k+1}$ 
9:   end if
10:   $k \leftarrow k + 1$ 
11: end loop
```

Notação (ver 2.1): com $\varphi(\alpha) = f(\mathbf{x}_k - \alpha \mathbf{g}_k)$, $\min_{\alpha > 0} \varphi(\alpha)$ é o *valor mínimo*; $\arg \min_{\alpha > 0} \varphi(\alpha)$ é o α **onde ele é atingido** (o minimizante).

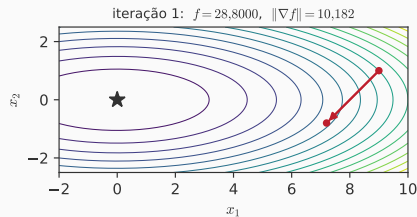
gradiente $\rightarrow$ direção $-\mathbf{g}_k$
 $\rightarrow$ pesquisa em linha (α_k) $\rightarrow$ novo ponto
 $\rightarrow$ paragem?

- **Custo por iteração:** $n_g = 1$ ($2n$ avaliações de f se o gradiente for numérico) + as avaliações de f da pesquisa em linha.
- **Memória:** só $\mathbf{x}_k$ e $\mathbf{g}_k$ — $O(n)$.
- Pesquisa exata: $\mathbf{g}_{k+1}^T \mathbf{g}_k = 0$. **Na prática** usa-se normalmente pesquisa em linha *inexata*, que exige apenas decréscimo suficiente (por exemplo, condições de Armijo/Wolfe).

O percurso em ziguezague

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ a partir de $(9, 1)$, pesquisa em linha exata.

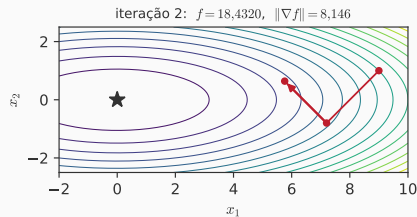
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9,0000; 1,0000)	45,00000	12,7279	—
1	(7,2000; -0,8000)	28,80000	10,1823	0,2000



O percurso em ziguezague

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ a partir de $(9, 1)$, pesquisa em linha exata.

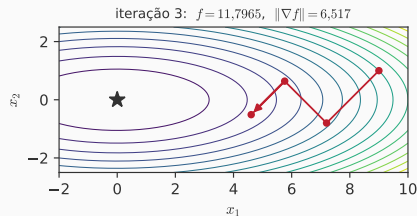
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9,0000; 1,0000)	45,00000	12,7279	—
1	(7,2000; -0,8000)	28,80000	10,1823	0,2000
2	(5,7600; 0,6400)	18,43200	8,1459	0,2000



O percurso em ziguezague

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ a partir de $(9, 1)$, pesquisa em linha exata.

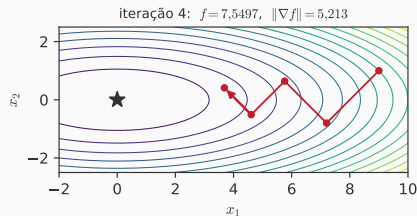
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9,0000; 1,0000)	45,00000	12,7279	—
1	(7,2000; -0,8000)	28,80000	10,1823	0,2000
2	(5,7600; 0,6400)	18,43200	8,1459	0,2000
3	(4,6080; -0,5120)	11,79648	6,5167	0,2000



O percurso em ziguezague

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ a partir de $(9, 1)$, pesquisa em linha exata.

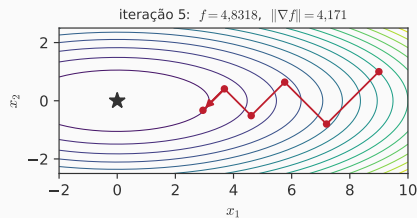
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9,0000; 1,0000)	45,00000	12,7279	—
1	(7,2000; -0,8000)	28,80000	10,1823	0,2000
2	(5,7600; 0,6400)	18,43200	8,1459	0,2000
3	(4,6080; -0,5120)	11,79648	6,5167	0,2000
4	(3,6864; 0,4096)	7,54975	5,2134	0,2000



O percurso em ziguezague

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ a partir de $(9, 1)$, pesquisa em linha exata.

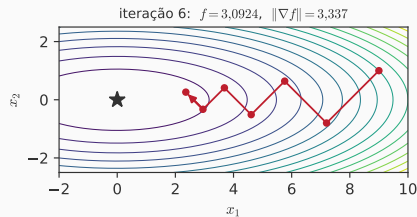
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9,0000; 1,0000)	45,00000	12,7279	—
1	(7,2000; -0,8000)	28,80000	10,1823	0,2000
2	(5,7600; 0,6400)	18,43200	8,1459	0,2000
3	(4,6080; -0,5120)	11,79648	6,5167	0,2000
4	(3,6864; 0,4096)	7,54975	5,2134	0,2000
5	(2,9491; -0,3277)	4,83184	4,1707	0,2000



O percurso em ziguezague

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ a partir de $(9, 1)$, pesquisa em linha exata.

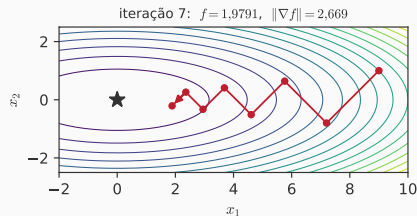
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9,0000; 1,0000)	45,00000	12,7279	—
1	(7,2000; -0,8000)	28,80000	10,1823	0,2000
2	(5,7600; 0,6400)	18,43200	8,1459	0,2000
3	(4,6080; -0,5120)	11,79648	6,5167	0,2000
4	(3,6864; 0,4096)	7,54975	5,2134	0,2000
5	(2,9491; -0,3277)	4,83184	4,1707	0,2000
6	(2,3593; 0,2621)	3,09238	3,3365	0,2000



O percurso em ziguezague

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ a partir de $(9, 1)$, pesquisa em linha exata.

k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9,0000; 1,0000)	45,00000	12,7279	—
1	(7,2000; -0,8000)	28,80000	10,1823	0,2000
2	(5,7600; 0,6400)	18,43200	8,1459	0,2000
3	(4,6080; -0,5120)	11,79648	6,5167	0,2000
4	(3,6864; 0,4096)	7,54975	5,2134	0,2000
5	(2,9491; -0,3277)	4,83184	4,1707	0,2000
6	(2,3593; 0,2621)	3,09238	3,3365	0,2000
7	(1,8874; -0,2097)	1,97912	2,6692	0,2000

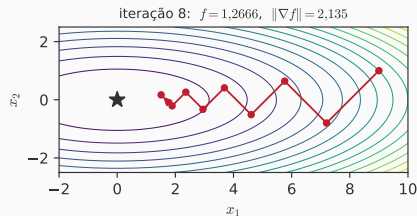


O percurso em ziguezague

$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ a partir de $(9, 1)$, pesquisa em linha exata.

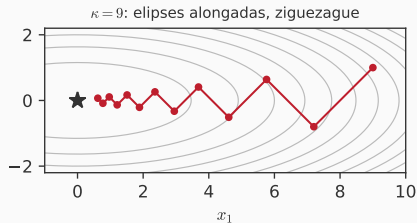
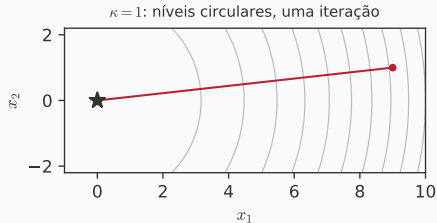
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $	α_{k-1}
0	(9,0000; 1,0000)	45,00000	12,7279	—
1	(7,2000; -0,8000)	28,80000	10,1823	0,2000
2	(5,7600; 0,6400)	18,43200	8,1459	0,2000
3	(4,6080; -0,5120)	11,79648	6,5167	0,2000
4	(3,6864; 0,4096)	7,54975	5,2134	0,2000
5	(2,9491; -0,3277)	4,83184	4,1707	0,2000
6	(2,3593; 0,2621)	3,09238	3,3365	0,2000
7	(1,8874; -0,2097)	1,97912	2,6692	0,2000
8	(1,5099; 0,1678)	1,26664	2,1354	0,2000

Cada iteração multiplica f por 0,64 e o passo é sempre $\alpha = 0,2$: o percurso é geometricamente semelhante de iteração para iteração. São precisas **74 iterações** para $\|\nabla f\| < 10^{-6}$ — numa quadrática de duas variáveis.



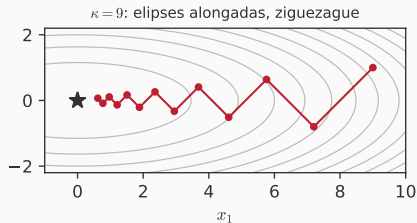
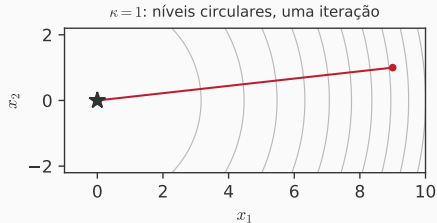
A forma das curvas de nível decide

Os passos são sempre perpendiculares; o que decide se isso custa caro é a **forma** das curvas de nível.



A forma das curvas de nível decide

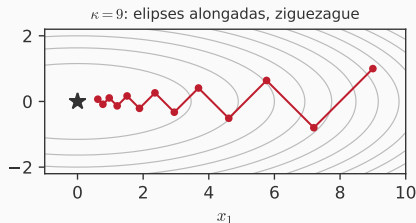
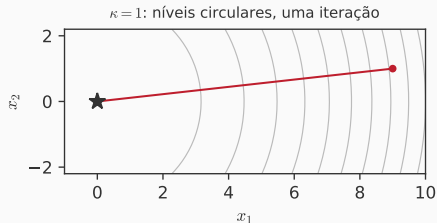
Os passos são sempre perpendiculares; o que decide se isso custa caro é a **forma** das curvas de nível.



Porque é que, com círculos, basta uma iteração?

A forma das curvas de nível decide

Os passos são sempre perpendiculares; o que decide se isso custa caro é a **forma** das curvas de nível.



Porque é que, com círculos, basta uma iteração? Com círculos ($\kappa = 1$), a normal à curva de nível passa pelo centro: $-\nabla f$ aponta **para o mínimo**. Com elipses 3 : 1 ($\kappa = \lambda_{\max}/\lambda_{\min}$ da Hessiana: aqui 9/1; ver página seguinte), aponta para a “parede” oposta, e os passos de 90° vão de parede em parede: 74 iterações.

Condicionamento: porque é que alongamento = lentidão

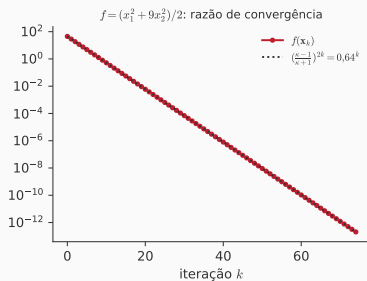
Hipóteses: $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$ com A **simétrica definida positiva**; método do gradiente com pesquisa em linha **exata**. Seja $\mathbf{x}^*$ o minimizante e $E(\mathbf{x}) = f(\mathbf{x}) - f^* = \frac{1}{2} \|\mathbf{x} - \mathbf{x}^*\|_A^2$, com $\|\mathbf{v}\|_A^2 = \mathbf{v}^T A \mathbf{v}$.

Cota (Kantorovich)

$$E(\mathbf{x}_{k+1}) \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^2 E(\mathbf{x}_k), \quad \kappa = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)},$$

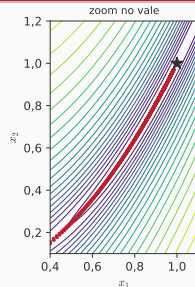
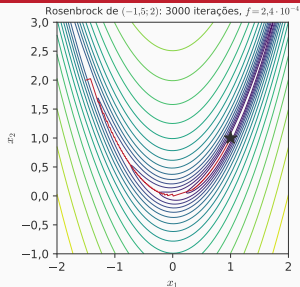
isto é, $\|\mathbf{x}_{k+1} - \mathbf{x}^*\|_A \leq \frac{\kappa - 1}{\kappa + 1} \|\mathbf{x}_k - \mathbf{x}^*\|_A$: fator $\left(\frac{\kappa - 1}{\kappa + 1} \right)^2$ no erro **em f** , fator $\frac{\kappa - 1}{\kappa + 1}$ no erro **na norma A** .

Aqui $\kappa = 9$, $f^* = 0$: a cota dá 0,64 para f , atingida em todas as iterações a partir de (9, 1). $\kappa = 1$: uma iteração; $\kappa = 100$: 0,96 — centenas de iterações. Convergência **linear**, lenta se κ é grande.



É uma cota de pior caso; noutras quadráticas pode não haver igualdade em cada iteração. Os semieixos das elipses de nível estão na razão $\sqrt{\kappa}$.

Rosenbrock: chegar ao vale não é o mesmo que percorrê-lo



- Chega ao vale em poucas iterações (o gradiente é grande e aponta para lá) e depois **arrasta-se**: ao fim de 3000 iterações ainda $f = 2,4 \cdot 10^{-4}$, $\mathbf{x} \approx (0,98; 0,97)$.
- No mínimo $(1, 1)$ a Hessiana tem $\kappa \approx 2,5 \cdot 10^3$: com pesquisa em linha exata, gradientes consecutivos ortogonais e progresso muito lento ao longo do vale.
- Mesmo assim, é um gradiente e uma pesquisa em linha por iteração: com n muito grande, os métodos de primeira ordem tornam-se atraentes pelo baixo custo e memória — é o ponto de partida do treino de redes neurais (“gradient descent”).

O núcleo de uma iteração, em MATLAB:

```
g = grad(x);           %  $n_g = 1$   
d = -g;                % direcao  
phi = @(a) f(x + a*d); % pesquisa em linha  
alpha = fminbnd(phi, 0, amax); % cap. 2  
% (se alpha ~ amax, alargar amax)  
x = x + alpha*d;
```

fminbnd usa secção áurea + interpolação parabólica (2.2–2.3).

- No exemplo ($\varepsilon_g = 10^{-6}$): 74 iterações, $n_g = 75$, cerca de 6 avaliações de f por pesquisa em linha com este fminbnd (com a pesquisa de alta precisão das figuras, cerca de 24).
- Sem gradiente analítico, cada gradiente por diferenças centrais custa $n_f = 2n$ (2.6).
- As funções completas (SteepestDescent, com info.nfev e info.ngcv; GradFD) estão no código complementar da UC.

O preço do método do gradiente

Vantagens

- só precisa de ∇f (ordem 1);
- barato por iteração; memória $O(n)$;
- simples; é a base de muitos métodos para n muito grande.

Limitações

- ziguezague: gradientes consecutivos ortogonais;
- convergência linear, muito lenta se κ é grande;
- ignora a curvatura: em vales estreitos aponta para a parede, não para o fundo.

“Convergência global” de um algoritmo local

Global = a partir de *qualquer* ponto inicial (sob as hipóteses: conjunto de nível limitado, gradiente Lipschitz (R), pesquisa em linha adequada) obtém-se aproximação à *estacionariedade* ($\|\nabla f(\mathbf{x}_k)\| \rightarrow 0$, ou pelo menos $\liminf \|\nabla f(\mathbf{x}_k)\| = 0$, consoante as hipóteses): a garantia é de estacionariedade, não necessariamente de convergência dos iterandos para um único ponto. **Não significa convergência para o mínimo global** — no cap. 5, “global” passa a referir-se ao ótimo.

O QUE DEVE FICAR — 3.3.1

1. Só com ∇f : $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$, a descida **mais íngreme localmente**; o passo α_k vem da pesquisa em linha (cap. 2).
2. Com pesquisa em linha exata, $\mathbf{g}_{k+1}^T \mathbf{g}_k = 0$: gradientes consecutivos **ortogonais** — o ziguezague.
3. Em quadráticas SPD, com pesquisa em linha exata, o erro em f satisfaz $E_{k+1} \leq \left(\frac{\kappa-1}{\kappa+1}\right)^2 E_k$: convergência **linear**, que se degrada quando κ cresce.
4. Custo baixo por iteração: um gradiente e uma pesquisa em linha; memória $O(n)$.
5. “Convergência global” = estacionariedade a partir de qualquer $\mathbf{x}_0$, sob hipóteses — **não** o mínimo global.

O gradiente usa só a **inclinação** e sofre com a geometria. Duas perguntas:

Podemos usar a **curvatura**? $\rightarrow$ Newton (3.3.2)

Podemos usar **memória** sem pagar a Hessiana? $\rightarrow$ gradientes conjugados (3.3.3)

Referências Lista completa: *Bibliografia da UC*, no fim do PDF da UC.

- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2.^a ed., Springer, 2006 — cap. 3.
- Fletcher, R., *Practical Methods of Optimization*, 2.^a ed., Wiley, 1987.

Recursos

- Código complementar: `SteepestDescent` (`info.nfev`, `info.ngev`), `GradFD`. MATLAB: `fminbnd` (pesquisa em linha).